

Taller #3

David Vallejo, Pablo Leon, Juan Sebastián Quintero

Facultad de Ingeniería

Pontificia Universidad Javeriana

Estructuras de datos

9 de octubre de 2026

1. Descripción general del problema

En este taller se desarrolla JaveBolsa, un programa que simula el funcionamiento básico de una bolsa de valores, permitiendo registrar órdenes de compra y venta de una acción. Cada orden tiene un precio y puede permanecer en espera hasta que llegue otra orden con la que pueda realizarse una negociación.

Para manejar estas órdenes se utiliza un árbol AVL, el cual permite almacenar los precios de forma ordenada y mantener la estructura balanceada mediante rotaciones. Las compras se representan con valores positivos y las ventas con valores negativos, lo que permite trabajar con ambos tipos de órdenes dentro de un mismo árbol.

El programa recibe las instrucciones desde un archivo de texto y las procesa en el orden en que aparecen. Dependiendo de cada evento, puede insertar una orden, eliminarla, realizar un cruce entre una compra y una venta o consultar el estado actual del árbol.

2. Diseño de los TAD

Para el desarrollo de JaveBolsa se consideran dos TAD: Orden y ArbolAVL. El primero representa la información de las órdenes de compra y venta, mientras que el segundo permite almacenar y organizar los precios de las órdenes que todavía se encuentran pendientes.

El árbol AVL es la estructura principal del programa, ya que permite realizar operaciones de inserción, eliminación y búsqueda, manteniendo los elementos ordenados y el árbol balanceado.

2.1. TAD Orden

El TAD Orden representa una orden de compra o venta registrada en el sistema. Cada orden cuenta con un identificador y un precio, que se representa mediante un número entero con signo. Los valores positivos corresponden a las compras y los negativos a las ventas.

Tabla 1. Diseño del TAD Orden

Elemento	Descripción
Nombre	Orden
Responsabilidad	Representar la información de una orden de compra o venta.
Estructura interna	Identificador de tipo cadena de texto y precio con signo de tipo entero.
Operación	Crear una orden con su identificador y precio.
Precondición	Recibir un identificador y un precio válido, con el signo correspondiente al tipo de orden.
Poscondición	La orden queda representada con el identificador y el precio recibidos.
Complejidad	O(k), considerando la copia de un identificador de longitud k.
Relación con otros TAD	El precio de la orden se utiliza en las operaciones del árbol AVL.

En el funcionamiento del programa, el identificador permite reconocer la orden que llega, mientras que el precio se utiliza para determinar si puede realizarse un cruce con otra orden pendiente.

2.2. TAD ArbolAVL

El TAD ArbolAVL es la estructura encargada de almacenar los precios de las órdenes que todavía no han sido ejecutadas. Su funcionamiento se basa en un árbol binario de búsqueda, donde los valores menores se ubican a la izquierda y los mayores a la derecha.

La diferencia principal frente a un árbol binario de búsqueda convencional es que el AVL mantiene su equilibrio mediante rotaciones después de realizar inserciones o eliminaciones. Para esto, cada nodo almacena su altura, permitiendo calcular el factor de equilibrio.

Tabla 2. Estructura interna del TAD ArbolAVL

Elemento	Descripción
Nombre	ArbolAVL
Responsabilidad	Almacenar y organizar los precios de las órdenes pendientes, permitiendo realizar búsquedas, inserciones, eliminaciones y consultas.
Raíz	Puntero que señala el nodo principal del árbol.
Valor	Número entero con signo que representa el precio de una orden.
Altura	Almacena la altura de cada nodo para comprobar el equilibrio del árbol.
Hijo izquierdo	Puntero al subárbol que contiene los valores menores.
Hijo derecho	Puntero al subárbol que contiene los valores mayores.
Relación con otros TAD	Utiliza los precios de las órdenes procesadas por el programa y una lista de enteros para devolver el recorrido inorden.

El campo altura es fundamental para el funcionamiento del AVL, ya que permite identificar cuándo un nodo se encuentra desbalanceado.

El factor de equilibrio se calcula restando la altura del subárbol derecho a la altura del izquierdo. Para que un nodo esté balanceado, este resultado debe encontrarse entre -1 y 1. Cuando se sale de ese rango, se realizan las rotaciones necesarias para recuperar el equilibrio.

2.3. Operaciones del TAD ArbolAVL

El árbol AVL cuenta con diferentes operaciones que permiten manejar las órdenes pendientes. Algunas modifican su estructura, como insertar y eliminar, mientras que otras permiten consultar la información almacenada sin realizar cambios.

Tabla 3. Operaciones del TAD ArbolAVL

Operación	Precondición	Poscondición	Complejidad
insertar(valor)	El valor no debe estar repetido.	Se inserta el valor y el árbol permanece balanceado.	O(log n)
eliminar(valor)	Recibir el valor que se desea eliminar.	Si el valor existe, se elimina y el árbol se rebalancea.	O(log n)
buscar(valor)	Recibir un valor entero.	Retorna verdadero si existe y falso en caso contrario.	O(log n)
mejorCompra()	Ninguna.	Retorna la compra de mayor precio o indica que no existe.	O(log n)
mejorVenta()	Ninguna.	Retorna el valor negativo más cercano a cero o indica que no existe.	O(log n)
recorridoInorden()	Ninguna.	Retorna los valores almacenados en orden ascendente.	O(n)

Las operaciones de inserción, eliminación y búsqueda tienen una complejidad de O(log n), debido a que el árbol se mantiene balanceado y no es necesario recorrer todos sus nodos.

Las consultas mejorCompra y mejorVenta también tienen complejidad O(log n), ya que aprovechan el orden del árbol para encontrar el valor correspondiente.

Por otro lado, el recorrido inorden tiene una complejidad O(n), debido a que necesita visitar todos los nodos para obtener los valores almacenados.

2.4. TAD Orden

El TAD Orden permite representar cada orden que ingresa al sistema mediante un identificador y un precio con signo. Las compras se representan mediante valores positivos y las ventas mediante valores negativos.

Este TAD se encuentra definido en el archivo Orden.h y se utiliza durante el procesamiento de los eventos para representar las órdenes antes de realizar las operaciones correspondientes en el árbol.

Tabla 4. Diseño del TAD Orden

Elemento	Descripción
Nombre	Orden
Responsabilidad	Representar una orden de compra o venta con su identificador y precio.
Identificador	Cadena de texto que permite identificar la orden.
Precio con signo	Número entero que representa el precio y permite diferenciar las compras de las ventas.
Operación	Orden(id, precio)
Precondición	Recibir un identificador y un precio con el signo correspondiente al tipo de orden.

Poscondición	Se crea una orden con los datos recibidos.
Complejidad	O(k), donde k corresponde a la longitud del identificador que se copia.
Relación con otros TAD	Proporciona el precio con signo que se utiliza en las operaciones del ArbolAVL.

2.5. Relación entre los TAD

Los dos TAD cumplen funciones diferentes dentro del diseño de JaveBolsa. Orden representa la información de las compras y ventas, mientras que ArbolAVL permite almacenar y organizar los precios de aquellas órdenes que todavía no han sido ejecutadas.

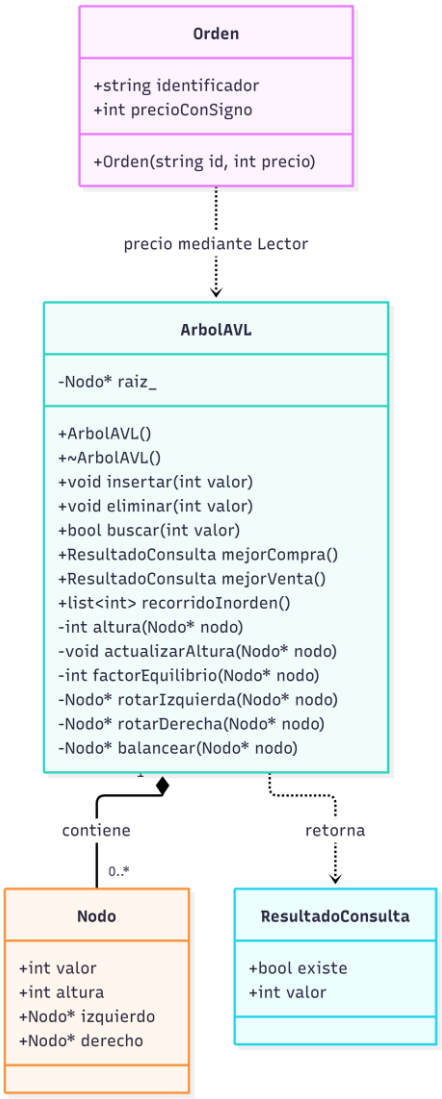
En la estructura implementada, el árbol almacena únicamente los precios con signo y no los objetos Orden completos. Esto permite identificar las compras y las ventas mediante valores positivos y negativos.

El procesamiento de las instrucciones se realiza en el archivo Lector.cxx, donde se determina si una orden puede cruzarse con otra o si debe permanecer almacenada en el árbol. De esta manera, las decisiones relacionadas con las compras y ventas se manejan por separado de las operaciones propias del AVL.

3. Diagrama de los TAD

El siguiente diagrama representa la estructura de los TAD utilizados en JaveBolsa, mostrando sus atributos, operaciones y relaciones. Se incluyen los TAD Orden y ArbolAVL, además de la estructura Nodo y el tipo ResultadoConsulta, que forman parte de la implementación del árbol.

El objetivo del diagrama es mostrar cómo se organiza la información y cuáles son las funciones que permiten manejar las órdenes pendientes.



3.1. Descripción del diagrama

En el diagrama se observa que el TAD Orden contiene el identificador y el precio con signo de cada orden. Esta información se utiliza durante el procesamiento de los eventos para determinar si una compra o venta debe ejecutarse o quedar pendiente.

Por su parte, ArbolAVL contiene las operaciones necesarias para insertar, eliminar, buscar y consultar los valores almacenados. Internamente utiliza nodos que guardan un valor, su altura y las referencias a sus hijos izquierdo y derecho.

También se utiliza la estructura ResultadoConsulta, que permite indicar si existe una compra o venta disponible y devolver su valor correspondiente.

De esta manera, cada estructura tiene una función específica y el árbol puede encargarse de mantener los valores ordenados y balanceados, sin manejar directamente las decisiones de compra y venta.

4. Funcionamiento del árbol AVL

El árbol AVL permite almacenar los precios de las órdenes pendientes de forma ordenada. Para esto, utiliza las propiedades de un árbol binario de búsqueda y un mecanismo de balanceo que evita que una de sus ramas crezca demasiado respecto a la otra.

A continuación, se explican las principales operaciones implementadas.

4.1. Inserción de elementos

La operación insertar(valor) permite agregar un nuevo precio al árbol. Primero se compara el valor con los nodos existentes para determinar su posición. Si es menor que el nodo actual, se continúa por la izquierda; si es mayor, por la derecha.

Cuando se encuentra una posición vacía, se crea un nodo con el valor recibido y altura inicial de 1. Después, al regresar hacia la raíz, se actualizan las alturas de los nodos y se comprueba su factor de equilibrio.

Si algún nodo queda desbalanceado, se realiza la rotación correspondiente para mantener las propiedades del árbol AVL.

4.2. Eliminación de elementos

La operación eliminar(valor) permite retirar un precio del árbol cuando una orden se cancela o se realiza un cruce entre una compra y una venta.

Para eliminar un nodo se consideran tres casos:

Si el nodo no tiene hijos, se elimina directamente.

Si tiene un solo hijo, este ocupa su posición.

Si tiene dos hijos, se reemplaza su valor por el de su sucesor inorden, que corresponde al menor valor del subárbol derecho. Después se elimina el sucesor de su posición original.

Una vez realizada la eliminación, se actualizan las alturas y se revisa el equilibrio de los nodos afectados. Esto es importante porque una eliminación puede provocar desbalances en varios niveles del árbol.

4.3. Balanceo y rotaciones

El balanceo permite que el árbol mantenga una altura adecuada y que las operaciones no tengan que recorrer una cantidad innecesaria de nodos.

Para comprobarlo, se calcula el factor de equilibrio, que corresponde a la diferencia entre la altura del subárbol izquierdo y la del derecho. Cuando este valor es mayor que 1 o menor que -1, se debe realizar una rotación.

Se utilizan cuatro casos de rotación:

Rotación	Descripción
Simple derecha (LL)	Se presenta cuando el desbalance ocurre hacia la izquierda del hijo izquierdo.
Simple izquierda (RR)	Se presenta cuando el desbalance ocurre hacia la derecha del hijo derecho.
Doble izquierda-derecha (LR)	Se realiza primero una rotación a la izquierda sobre el hijo izquierdo y después una rotación a la derecha.
Doble derecha-izquierda (RL)	Se realiza primero una rotación a la derecha sobre el hijo derecho y después una rotación a la izquierda.

Estas rotaciones reorganizan los nodos sin modificar el orden de sus valores. De esta manera, el árbol continúa funcionando como un árbol binario de búsqueda, pero conserva su equilibrio.

4.4. Búsqueda de elementos

La operación buscar(valor) permite comprobar si un precio se encuentra almacenado en el árbol.

La búsqueda comienza desde la raíz y compara el valor solicitado con el nodo actual. Si son iguales, retorna verdadero. Si el valor buscado es menor, continúa por el subárbol izquierdo; si es mayor, continúa por el derecho.

Si llega a una posición vacía sin encontrar el valor, retorna falso.

4.5. Consulta de la mejor compra

La operación mejorCompra() permite encontrar el precio más alto que un comprador está dispuesto a pagar.

Como el árbol almacena los valores de manera ordenada, el valor máximo se encuentra en el nodo ubicado más a la derecha. Por esta razón, la búsqueda consiste en avanzar por los hijos derechos hasta encontrar el último nodo.

Si el valor encontrado es positivo, corresponde a la mejor compra disponible. En caso de que el árbol esté vacío o el valor máximo no sea positivo, se indica que no existen compras pendientes.

4.6. Consulta de la mejor venta

La operación mejorVenta() permite encontrar el precio más bajo por el que un vendedor está dispuesto a vender.

Las ventas se almacenan utilizando valores negativos, por lo que la mejor venta corresponde al valor negativo más cercano a cero.

Para encontrarlo, se recorre el árbol comparando los nodos con cero. Cuando se encuentra un valor negativo, se guarda como posible resultado y se continúa hacia la derecha para buscar otro negativo más cercano a cero. Cuando el valor es positivo, se continúa por la izquierda.

Al finalizar, se retorna el mejor candidato encontrado. Si no se encontró ningún valor negativo, significa que no hay ventas pendientes.

Diferencia entre mejorCompra() y mejorVenta()

Aunque ambas operaciones buscan la mejor oferta disponible, su recorrido es diferente.

En mejorCompra() basta con encontrar el valor máximo del árbol, ya que las compras se representan mediante números positivos y el precio más alto siempre se encuentra hacia el extremo derecho.

En cambio, mejorVenta() no puede buscar simplemente el valor mínimo del árbol, porque eso devolvería el número más negativo, que representa la venta de mayor precio y no la más barata.

Por ejemplo, si se encuentran almacenados los valores -80, -60 y -50, la mejor venta sería -50, porque representa un precio real de 50.

Por esta razón, mejorVenta() necesita encontrar el valor negativo más cercano a cero, mientras que mejorCompra() busca el valor máximo del árbol.

4.7. Recorrido inorden

La operación recorridoInorden() permite consultar todos los valores almacenados en el árbol siguiendo un orden ascendente.

Para realizar el recorrido, primero se visita el subárbol izquierdo, después el nodo actual y finalmente el subárbol derecho.

Como resultado, se obtiene una lista ordenada que incluye tanto las ventas, representadas con valores negativos, como las compras, representadas con valores positivos.

Este recorrido permite observar el estado de las órdenes pendientes sin modificar la estructura del árbol.

5. Plan de pruebas

Para comprobar el funcionamiento del programa se realizaron diferentes pruebas sobre las operaciones del árbol AVL. Estas permiten verificar que los valores se inserten, eliminen y consulten correctamente, y que el árbol mantenga su equilibrio después de modificar su estructura.

También se realizaron pruebas para los cuatro tipos de rotaciones, los diferentes casos de eliminación y el procesamiento de las órdenes de compra y venta.

5.1. Pruebas de inserción y rotaciones

Las primeras pruebas se enfocaron en la inserción de elementos. Para cada una se utilizó una secuencia de valores que provocara un tipo específico de desbalance y permitiera comprobar la rotación correspondiente.

Tabla 5. Pruebas de inserción y rotaciones

Función	Entrada o estado previo	Resultado esperado	Resultado obtenido
insertar() – LL	Insertar 30, 20, 10.	Rotación simple a la derecha. Raíz 20.	Raíz 20. Recorrido: 10, 20, 30.
insertar() – RR	Insertar 10, 20, 30.	Rotación simple a la izquierda. Raíz 20.	Raíz 20. Recorrido: 10, 20, 30.
insertar() – LR	Insertar 30, 10, 20.	Rotación doble izquierda-derecha. Raíz 20.	Raíz 20. Recorrido: 10, 20, 30.
insertar() – RL	Insertar 10, 30, 20.	Rotación doble derecha-izquierda. Raíz 20.	Raíz 20. Recorrido: 10, 20, 30.

En los cuatro casos se obtuvo la raíz esperada y se comprobó que los valores continuaran ordenados. También se verificaron las alturas y los factores de equilibrio de los nodos después de cada inserción.

5.2. Pruebas de eliminación y balanceo

Para comprobar la eliminación se utilizaron diferentes árboles iniciales. En cada caso se eliminó un valor que provocara un desbalance, con el fin de revisar si se aplicaba la rotación necesaria.

Tabla 6. Pruebas de eliminación y rotaciones

Función	Entrada o estado previo	Resultado esperado	Resultado obtenido
eliminar() – LL	Insertar 30, 20, 40, 10, 25. Eliminar 40.	Rotación simple derecha. Raíz 20.	Raíz 20. Recorrido: 10, 20, 25, 30.
eliminar() – RR	Insertar 20, 10, 30, 25, 40. Eliminar 10.	Rotación simple izquierda. Raíz 30.	Raíz 30. Recorrido: 20, 25, 30, 40.
eliminar() – LR	Insertar 30, 10, 40, 20. Eliminar 40.	Rotación doble izquierda-derecha. Raíz 20.	Raíz 20. Recorrido: 10, 20, 30.
eliminar() – RL	Insertar 10, 5, 30, 20. Eliminar 5.	Rotación doble derecha-izquierda. Raíz 20.	Raíz 20. Recorrido: 10, 20, 30.

Los resultados muestran que el árbol recuperó su equilibrio después de eliminar los valores indicados. En todos los casos se conservó el orden de los elementos y se obtuvieron las raíces esperadas.

5.3. Pruebas de las operaciones del árbol

Además de las rotaciones, se comprobó el funcionamiento de las operaciones principales del árbol.

Para las primeras pruebas de eliminación se utilizó un árbol con los valores 20, 10, 30, 5, 15, 25 y 40. Las eliminaciones se realizaron de manera consecutiva para comprobar los diferentes casos.

Tabla 7. Pruebas de las operaciones del ArbolAVL

Función	Entrada o estado previo	Resultado esperado	Resultado obtenido
Constructor y consultas	Crear un árbol vacío.	No debe contener elementos ni mejores ofertas.	Árbol vacío y consultas sin resultados.
eliminar()	Eliminar el nodo hoja 5.	El valor 5 desaparece del árbol.	Recorrido: 10, 15, 20, 25, 30, 40.
eliminar()	Eliminar 10 después del caso anterior.	El nodo 15 ocupa su posición.	Recorrido: 15, 20, 25, 30, 40.
eliminar()	Eliminar 30, que tiene dos hijos.	Se utiliza su sucesor inorden.	Recorrido: 15, 20, 25, 40.
eliminar()	Intentar eliminar 99, que no existe.	El árbol no debe modificarse.	Se conservaron los valores anteriores.
buscar()	Buscar 25 y 99.	Verdadero para 25 y falso para 99.	Verdadero y falso, respectivamente.
mejorCompra()	Árbol con 15, 20, 25 y 40.	Retornar 40.	Retornó 40.
mejorVenta()	Árbol con valores positivos únicamente.	Indicar que no hay ventas.	No se encontró ninguna venta.
recorridoInorden()	Árbol con 15, 20, 25 y 40.	Retornar los valores en orden ascendente.	15, 20, 25, 40.
mejorCompra()	Árbol con -55, -53, -60, 50 y 52.	Retornar 52.	Retornó 52.
mejorVenta()	Árbol con -55, -53, -60, 50 y 52.	Retornar -53.	Retornó -53.
recorridoInorden()	Árbol con -55, -53, -60, 50 y 52.	Mostrar los valores ordenados.	-60, -55, -53, 50, 52.
mejorCompra() y mejorVenta()	Árbol con -30, -20 y -10.	No hay compras y la mejor venta es -10.	Compra inexistente y mejor venta -10.
insertar() y eliminar()	Insertar los números del 1 al 300 y eliminarlos en el mismo orden.	Mantener el equilibrio durante las operaciones y terminar con el árbol vacío.	Se mantuvo el equilibrio y el árbol terminó vacío.
Funciones auxiliares	Árbol con 20, 10, 30, 5, 15, 25 y 40.	Altura de la raíz igual a 3, factor de equilibrio 0 y mínimo 5.	Se obtuvieron los tres valores esperados.

Estas pruebas permitieron comprobar diferentes situaciones, incluyendo árboles vacíos, eliminación de nodos con distinta cantidad de hijos y consultas con valores positivos y negativos.

También se revisaron las funciones auxiliares utilizadas para calcular las alturas y mantener el equilibrio del árbol.

5.4. Pruebas del TAD Orden y procesamiento de eventos

Finalmente, se comprobó la creación de órdenes y el procesamiento del archivo de entrada.

Tabla 8. Pruebas del sistema JaveBolsa

Función	Entrada o estado previo	Resultado esperado	Resultado obtenido
Constructor de Orden	Crear una compra con identificador c1 y precio 50.	Guardar el identificador c1 y el precio 50.	Se almacenaron ambos valores correctamente.
Constructor de Orden	Crear una venta con identificador v1 y precio -55.	Guardar el identificador v1 y el precio -55.	Se almacenaron ambos valores correctamente.
procesarEventos()	Ejecutar el archivo eventos.txt proporcionado en el taller.	Procesar las compras, ventas, cruces y cancelaciones en orden.	Se procesaron los eventos y se realizaron los cruces esperados.
CONSULTAR	Consultar el árbol después de procesar todos los eventos del ejemplo.	Mejor compra: ninguna. Mejor venta: 55. Recorrido: -55.	Mejor compra: ninguna. Mejor venta: 55. Recorrido: -55.

Al finalizar el procesamiento del archivo, únicamente quedó pendiente la venta con precio 55, representada en el árbol mediante el valor -55. Este resultado coincide con el ejemplo presentado en el enunciado del taller.

5.5. Resultados generales

Las pruebas realizadas permitieron comprobar el funcionamiento de las operaciones principales del árbol AVL y las cuatro rotaciones necesarias para mantener su equilibrio.

También se verificó que las órdenes de compra y venta se procesaran de acuerdo con las condiciones establecidas en el problema. Los resultados obtenidos coincidieron con los esperados en los casos evaluados.

5.6. Pruebas en C++

Se implemento un programa que ejecuta las pruebas y compara los resultados con las salidas esperadas, los casos cubren inserción, eliminación (hoja un y dos hijos), búsqueda, consultas y árbol vacío:

Caso	Que se evaluó
1 insertar simple	insertar, mejorCompra, recorridoInorden
2 árbol vacío	Constructor y consultas
3 solo ventas sin compra	insertar y majorVenta
4 eliminar hoja	eliminar (hoja)
5 eliminar un hijo	eliminar (un hijo)
6 eliminar dos hijos	eliminar (dos hijos)
7 secuencia cruce completa	Flujo completo de eventos
8 cancelar inexistente	buscar, eliminar (inexistente)
9 mejor venta no es mínimo	mejorVenta

El código completo del plan de pruebas se adjunta a la entrega

6. Análisis de complejidad

El uso de un árbol AVL permite mantener los valores organizados y evitar que su estructura se desbalancee después de realizar inserciones o eliminaciones. Esto es importante porque el rendimiento de las operaciones depende principalmente de la altura del árbol.

6.1. Comportamiento de un árbol sin balancear

Si se utilizara un árbol binario de búsqueda convencional, su estructura podría desbalancearse dependiendo del orden en que lleguen los valores.

Por ejemplo, si se registran compras con precios de 10, 20, 30, 40 y 50, cada nuevo valor se insertaría a la derecha del anterior. Como resultado, se formaría una estructura similar a una lista enlazada, donde cada nodo tiene únicamente un hijo derecho.

En esta situación, las operaciones de inserción, búsqueda y eliminación podrían necesitar recorrer todos los nodos del árbol, pasando de una complejidad O(log n) a O(n) en el peor caso.

En JaveBolsa, esto podría ocurrir durante una racha de mercado en la que los precios de compra aumentan continuamente y las órdenes permanecen pendientes.

6.2. Ventajas del balanceo AVL

Para evitar este problema, el árbol AVL utiliza rotaciones que permiten reorganizar los nodos cuando se presenta un desbalance.

Después de insertar o eliminar un elemento, se actualizan las alturas de los nodos y se revisan sus factores de equilibrio. Si alguno queda fuera del rango permitido, se realizan las rotaciones correspondientes.

Gracias a este mecanismo, la altura del árbol se mantiene en O(log n), evitando que las operaciones tengan que recorrer una cadena de nodos demasiado larga.

6.3. Comparación de complejidades

Tabla 9. Comparación entre un árbol BST sin balancear y un árbol AVL

Operación	BST sin balancear (peor caso)	Árbol AVL (peor caso)
insertar(valor)	O(n)	O(log n)
eliminar(valor)	O(n)	O(log n)
buscar(valor)	O(n)	O(log n)
mejorCompra()	O(n)	O(log n)
mejorVenta()	O(n)	O(log n)
recorridoInorden()	O(n)	O(n)

Donde n representa la cantidad de nodos almacenados en el árbol.

Las operaciones insertar y eliminar mantienen una complejidad O(log n) en el AVL, incluso cuando es necesario realizar rotaciones para recuperar el equilibrio.

De igual manera, las consultas mejorCompra() y mejorVenta() aprovechan el orden de los elementos para encontrar el valor requerido siguiendo un camino dentro del árbol. Si este no estuviera balanceado, dicho camino podría llegar a tener una longitud de n nodos.

Por otra parte, el recorrido inorden mantiene una complejidad O(n) en ambas estructuras, debido a que necesita visitar todos los elementos almacenados.

6.4. Análisis aplicado a JaveBolsa

En el caso de JaveBolsa, mantener el árbol balanceado permite procesar las órdenes sin que el rendimiento dependa demasiado del orden en que llegan los precios.

Esto resulta útil porque el programa necesita consultar constantemente la mejor compra o venta antes de decidir si una nueva orden puede ejecutarse o debe quedar pendiente.

Si el árbol estuviera desbalanceado, estas consultas podrían volverse más costosas a medida que aumentara la cantidad de órdenes almacenadas. En cambio, el AVL permite mantener una complejidad logarítmica para estas operaciones.

Por esta razón, utilizar un árbol AVL es una buena alternativa para el problema planteado, ya que permite organizar los precios y realizar las operaciones necesarias manteniendo una altura controlada.

7. Conclusiones

Con el desarrollo de este taller se logró implementar un sistema básico de compra y venta de acciones utilizando un árbol AVL como estructura principal. Esta implementación permitió almacenar las órdenes pendientes mediante valores enteros con signo, realizar búsquedas, inserciones, eliminaciones y consultas, además de procesar los eventos recibidos desde un archivo de texto.

A partir de las pruebas realizadas, se comprobó la importancia de mantener el árbol balanceado después de insertar o eliminar elementos. Las rotaciones permiten conservar una altura controlada, evitando que las operaciones se vuelvan más costosas cuando aumenta la

cantidad de nodos. De esta manera, el taller permitió comprender mejor el funcionamiento de los árboles AVL y su aplicación en un problema donde es necesario mantener la información ordenada y realizar consultas constantemente.